Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 27

Hash Tables

Open Addressing (Ctd.)

Open Addressing

- Open addressing: Probe the hash table at a sequence of indices H_0(key), H_1(key), ...
  - Until reaching a location that has never been used
  - For inserting, first free location along the way is used (greedy)
  - In practice, probe sequence designed so that it reaches all indices in m steps
- Uniform Probing: For each (key,i), H_i(key) is independent random (unrealistic)
  - Expected Probe-sequence length: $\Theta\left(\frac{1}{1-\alpha}\right)$
  - Optimal for greedy, non-reordering probing strategy
- Linear Probing: $H_i(\text{key}) = H_0(\text{key}) + i \pmod m$
  - Expected Probe-sequence length: $\Theta\left(\frac{1}{(1-\alpha)^2}\right)$

Quadratic Probing

- Quadratic probing: $H_i(key) = H_0(key) + i^2 \pmod m$
  - Still simple: increment H_i by 2i+1 (increment the increment by 2)
  - Somewhat cache-friendly: Initially, for small i, inspected locations will mostly be in the cache
  - Expected probe-sequence length close to uniform probing for small α
- The above basic quadratic probe sequence can visit at most half of the indices (which happens when m is prime). Alternatives visit all indices:
  - When $m=2^k$, $H_i(key) = H_0(key) + i + 2\,i^2 \pmod m$ covers all indices. So does $H_i(key) = H_0(key) + i(i+1)/2 \pmod m$.
  - When m is a prime s.t. m-1 is not a multiple of 4, $H_0(key) \pm i^2 \pmod m$

Double-Hash Probing

- Double-Hash probing: $H_i(key) = H_0(key) + i \times H_\Delta(key) \pmod m$
  - Requires computing two hashes instead of one
  - Not cache friendly: Steps of size $H_\Delta(key)$ while probing
  - Probe-sequence length behaviour very close to that in uniform probing
- If $H_\Delta(key)$ is co-prime with m, this sequence reaches all m indices
  - E.g., $m=2^k$ and $H_\Delta(key) = 2H_\delta(key)+1$
  - Because 0, a, 2a, ..., (m-1)a (mod m) is a permutation of 0,1,...,m-1 whenever gcd(m,a)=1
  - Why? If $ai=aj \pmod m$, a(i-j) is a multiple of m. Since a has no factors of m, i-j has to be a multiple of m. Since $|i-j| < m$, we need i-j=0

Hybrid Probing

- We can consider a hybrid version that combines linear probing with another probing strategy h_0, h_1, ...
- Probe sequence: $T \cdot h_0(key)$, ... , $T \cdot h_0(key) + T\text{-}1$, $T \cdot h_1(key)$, ... , $T \cdot h_1(key) + T\text{-}1$, ...
- In modern hardware, can carry out a match on a small set of memory locations (T here) in one step
  - Match a one-byte *fingerprint* of the key (just another hash, typically implemented as a few reserved bits of h_0) against the fingerprints of the T locations. Look for a key match iff fingerprint matches.
  - Similarly, in one step can find a free slot out of the T slots
- Also, cache locality benefits of linear probing, along with probe-sequence length of the other probing strategy

Attendance Break!

Login using CSE LDAP id at

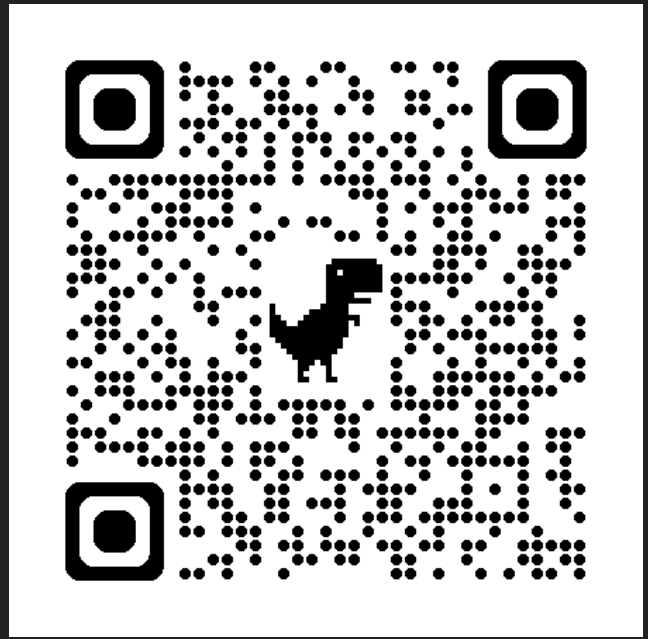
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Beating Uniform Probe

- For greedy probing, the best probe-sequence lengths were by uniform probe (or double-hash probe)
  - Expectation $\frac{1}{1-\alpha}$, and about 5% probes are over 3x the expectation
- Can we do better?
- Yes, by allowing reordering while probing!
- We'll see two schemes:
  - Robin Hood hashing: Take from the rich and give to the poor. Doesn't change the average, but virtually eliminates unlucky probes.
  - Cuckoo Hashing: Push out and occupy. Insert takes longer, but search is O(1), irrespective of the load.

Recent discovery: Also, possible without reordering, by not being greedy

Robin Hood Hashing

- Modified linear probing. Every item stores how far they are from their "home": `PSL(i) = i - h0(A[i].key) (mod m)`. The "rich" are close to their home.
- Inserting: Start linearly probing from the home index i_0, with `myPSL(i0)=0`.
  - As we are probing, the item we are holding gets poorer (`myPSL(i+1)=myPSL(i)+1`)
  - At an index i, if the item sitting there is richer than the item we are holding, (`PSL(i) < myPSL(i)`) then we swap the two. The richer item that is evicted continues down the linear probe path (getting poorer as it goes).
  - Continue till we find a free slot for the item we are holding.
- Searching: Stop when we encounter items richer than us.
- Deleting: Left shift each item to the right of the deleted item by one, till we encounter an empty slot or an item at its home. No tombstones!